Social Data Analytics
Task Work Report

Introduction

In this report we will be showing our work in the NLP field to classify movie reviews to the nature of them being positive, negative, or neutral, which will be accomplished through the use of modern techniques represented in text preprocessing, text representation, and machine learning algorithms, and together we will review the results in a visualised and simplified manner, so without further ado lets dive in the report.

Methodology

Data Source

First we decided on our topic to be categorization of movie reviews and so we needed to bring data that contains viewers reviews about some movies that at least contain two main columns (textual review and movie rating) as those were the main focus in this project, and the method we chose to obtain this data was by using TMDB api for movie reviews and we succeeded in obtaining approximately 5000 records each containing data like (movie_id, movie_title, review, rating, …).

Data Preparation

Our main purpose was to classify textual reviews into one of three categories (positive, negative, neutral) so we had to add a column to our data which will carry the classification (in our opinion) of each review so it could act as the true label for the dataset which will then be compared with the predicted labels from the models.

Data Preprocessing

In this stage, we applied a configurable and modular text preprocessing pipeline to clean and normalize the raw textual reviews, ensuring that the data becomes suitable for downstream machine learning tasks. The preprocessing process was implemented as a Python script with flexible command-line arguments, allowing us to experiment with different combinations of techniques depending on the modeling requirements.
The pipeline begins by handling noise commonly found in user-generated content. This includes removing HTML tags, URLs, mentions, hashtags, numerical values, and punctuation symbols.

Additionally, all text was converted to lowercase to ensure uniformity and reduce feature sparsity caused by case sensitivity.
To further refine the textual data, we removed stopwords using a predefined English stopword list, eliminating commonly occurring words that carry little semantic meaning. We also filtered out single-character tokens and excessive whitespace to improve the overall text quality.
For linguistic normalization, we incorporated lemmatization using lexical processing, which reduces words to their base or dictionary form. This step helps unify different inflections of the same word under a single representation. Furthermore, we applied spelling correction using an efficient dictionary-based algorithm (SymSpell), which improves the robustness of the dataset by correcting misspelled terms frequently found in informal reviews.
The preprocessing pipeline was designed to be highly configurable, enabling selective activation of each step such as stopword removal, lemmatization, or spelling correction. This flexibility allowed us to generate multiple cleaned versions of the dataset and evaluate their impact on model performance.
Finally, the processed reviews were stored in a new column (cleaned_review), and any empty or invalid entries resulting from preprocessing were removed to ensure data integrity.

Ground Truth Labeling

To accomplish this part we participated with three annotators two of them were from us and used the rating column as the third one (rating < 4 = negative, 4 < rating < 7 = neutral, rating > 7 = positive), after that we supported our labeling with the Cohen Kappa's test and got a result score of 77% which indicated high substantial agreement between the 3 annotators and in the end combined the three label columns into one by majority label per record.

Text Representation

After completing the preprocessing stage, we proceeded to transform the textual data into numerical representations suitable for machine learning models. As required, we utilized the preprocessing pipeline to generate three distinct schemas of the dataset, each reflecting a different level of text normalization. The first schema represents a fully cleaned version of the data, the second excludes lemmatization, and the third retains more of the raw textual characteristics by preserving punctuation, casing, and stopwords.
Using these three schemas, we constructed two different types of text representations to capture both statistical and semantic features of the text.
The first approach was based on Term Frequency–Inverse Document Frequency (TF-IDF), which captures the importance of words relative to the corpus. For each schema, a separate TF-IDF vectorizer was trained to account for the differences in preprocessing. The resulting sparse matrices were stored efficiently in .npz format, while the corresponding fitted vectorizers were serialized and saved as .pkl files to ensure reproducibility during deployment and inference.
The second approach relied on distributed word representations using pre-trained GloVe embeddings. We utilized the 100-dimensional embedding file (glove.6B.100d.txt) and integrated

it using the Gensim library. For each schema, textual data was converted into dense vector representations by aggregating the embeddings of individual words. The resulting feature sets were saved as .csv files for further use in model training.
In total, this process yielded six distinct text representations: three based on TF-IDF and three based on GloVe embeddings. This diversified representation strategy allowed us to systematically evaluate how different preprocessing levels and embedding techniques impact the performance of classification models.

Modelling

Lexicon-Based Models

In this part of the modelling phase, we implemented two lexicon-based approaches for sentiment classification: AFINN and VADER. These models rely on predefined sentiment dictionaries rather than learned parameters, making them suitable as baseline methods for evaluating the effectiveness of our preprocessing and representation strategies.
For the AFINN model, we utilized a word-level sentiment lexicon where each term is associated with an integer score representing its polarity. To enhance the basic implementation, we introduced several improvements to better capture contextual sentiment within the reviews. First, we applied a custom tokenization process to standardize the input text. Then, we incorporated a negation-handling mechanism using a sliding window approach, where the polarity of words following a negation term is inverted within a محدد range. Additionally, we accounted for intensifiers such as “very” and “extremely” by scaling the sentiment score of the associated words. These enhancements allowed the model to better reflect the semantic nuances present in natural language.
The final sentiment score for each review was computed as the sum of individual word contributions, after applying negation and intensity adjustments. Instead of relying on a simple zero threshold, we used tuned positive and negative thresholds to achieve more stable classification into three categories: positive, negative, and neutral.
For the VADER model, we used the Sentiment Intensity Analyzer provided by the NLTK library. VADER is specifically designed for social text and incorporates heuristics such as punctuation emphasis, capitalization, and degree modifiers. The model produces a compound score ranging from -1 to 1, which represents the overall sentiment of the text. Based on this score, we assigned sentiment labels using predefined thresholds to classify reviews into positive, negative, or neutral categories.
Both AFINN and VADER models were applied across all three dataset schemas generated during preprocessing. For each schema, predictions were generated using the corresponding cleaned reviews and then mapped into categorical sentiment labels. The outputs were saved as separate CSV files, each containing the ground truth labels alongside the predicted sentiments for further evaluation and comparison.
This approach resulted in six sets of predictions (two models across three schemas), enabling a comprehensive analysis of how lexicon-based methods perform under different preprocessing conditions.

Machine Learning Models

In addition to lexicon-based approaches, we implemented supervised machine learning models to learn sentiment patterns directly from the data. Specifically, we utilized Support Vector Machines (SVM) and Linear Regression models, applying them across all six text representations generated earlier (three TF-IDF and three GloVe-based representations).
To ensure consistency, each representation was paired with its corresponding schema labels. For TF-IDF representations stored in sparse .npz format, the feature matrices were loaded directly and aligned with the sentiment labels from the associated schema. For GloVe representations stored as dense .csv files, the features were combined with the sentiment labels before splitting into input variables and targets.
For both models, the dataset was divided into training and testing sets using a 75%-25% split with a fixed random state to ensure reproducibility.
The first model, Support Vector Machine (SVM), was trained using a balanced class weighting scheme to address any potential class imbalance in the dataset. The model was fitted on the training data and used to generate predictions on the test set. These predictions were then saved alongside the true labels for each representation, resulting in six separate output files corresponding to each feature configuration.
The second model, Linear Regression, was adapted for classification by mapping the categorical sentiment labels (negative, neutral, positive) into numerical values (-1, 0, 1). After training the regression model, predictions were produced as continuous values and subsequently converted back into discrete sentiment classes using predefined thresholds. This approach allowed us to approximate classification behavior using a regression framework. The predicted labels were then remapped to their original categorical form and saved for evaluation.
Overall, this stage produced twelve sets of predictions (two models across six representations), enabling a detailed comparison between different feature extraction techniques and learning algorithms. This setup allowed us to analyze how both traditional statistical representations (TF-IDF) and semantic embeddings (GloVe) interact with different model architectures in the context of sentiment classification.

Model Evaluation & Benchmarking

In this stage, we evaluated and benchmarked all the produced models (a total of 18 models) in order to identify the most promising combinations of preprocessing techniques, text representations, and modelling approaches for further optimization.
To ensure a consistent and fair comparison, we evaluated each model using the same test split generated during the modelling phase. The evaluation process was implemented using standard metrics from the Scikit-learn library, providing a comprehensive view of model performance across different dimensions.
The primary evaluation metrics used were accuracy, precision, recall, and F1-score. In particular, we relied on the weighted F1-score as a key metric since it accounts for class imbalance and provides a more reliable measure of overall performance compared to accuracy alone. Additionally, we generated full classification reports for each model, which include per-class precision, recall, and F1-scores, allowing for detailed inspection of how each class (positive, negative, neutral) was handled.

To further analyze model behavior, confusion matrices were computed for all models. These matrices provided insight into misclassification patterns, such as whether models tended to confuse neutral reviews with positive or negative ones, which is a common challenge in sentiment analysis tasks.
The evaluation pipeline was applied uniformly across all model outputs by comparing the predicted labels with the ground truth sentiment labels stored in the output files. This systematic approach ensured comparability between lexicon-based models (AFINN and VADER) and machine learning models (SVM and Linear Regression), as well as across all six text representations.
Through this benchmarking process, we were able to identify performance trends across different configurations, forming a solid foundation for selecting the best-performing models to proceed with in the optimization stage.

Model Optimization

Following the evaluation and benchmarking phase, we selected the most promising models for further optimization. Based on the observed performance, Support Vector Machine (SVM) models combined with TF-IDF representations demonstrated superior results compared to other approaches, particularly when applied to the fully cleaned schema and the without stopwords and punctuation schema. These configurations were therefore chosen for optimization and eventual deployment.
To improve model performance, we applied hyperparameter tuning using Grid Search with cross-validation. The optimization process focused primarily on the SVM model, as it consistently outperformed Linear Regression and lexicon-based methods during the evaluation stage.
For the SVM models, we explored multiple hyperparameters including the regularization parameter (C), kernel type, and gamma values. Initially, both linear and RBF kernels were considered; however, the optimization process emphasized the RBF kernel due to its ability to capture non-linear relationships in the data. The search space included different values for C (controlling the trade-off between margin maximization and classification error) and gamma (controlling the influence of individual training samples). The models were evaluated using cross-validation (5-fold) with F1-score (both macro and weighted variants) as the primary scoring metric, ensuring robustness against class imbalance.
In addition to SVM optimization, we conducted a limited hyperparameter search for the Linear Regression model. The tuning process included parameters such as intercept fitting and positivity constraints. However, despite optimization, Linear Regression did not achieve performance comparable to SVM, reinforcing the decision to prioritize SVM-based models.
To address potential overfitting and underfitting, we relied on cross-validation during Grid Search and used balanced class weighting in SVM. This helped ensure that the model generalized well across different data splits while maintaining fair treatment of all sentiment classes.
After optimization, we compared the performance of the models before and after tuning. The optimized SVM models showed noticeable improvements in F1-score, confirming the

effectiveness of hyperparameter tuning and kernel selection. As a result, two models were selected as final candidates for deployment:

- SVM with TF-IDF using the fully cleaned schema
- SVM with TF-IDF using the schema without stopword removal and punctuation normalization

These models provided the best balance between accuracy, generalization, and robustness, making them suitable for production deployment.

Error Analysis

After selecting the final models, we conducted an in-depth error analysis to understand the limitations of our system and identify patterns in misclassified samples. This step is critical for interpreting model behavior beyond numerical metrics and provides insight into areas where the model struggles.

We began by examining the incorrectly classified instances from the prediction outputs, comparing the true sentiment labels with the predicted ones. Special attention was given to confusion patterns observed in the confusion matrices, particularly cases where the model confused neutral reviews with positive or negative ones.

One of the most common sources of error was the ambiguity of neutral reviews. These reviews often contained a mix of positive and negative expressions, making it difficult for the model to assign a clear sentiment. As a result, many neutral samples were misclassified as either positive or negative, indicating that the boundary between classes is not always well-defined in natural language.

Another observed issue was the presence of contextual or implicit sentiment. For example, sarcastic or ironic statements were frequently misclassified because the model relies primarily on surface-level textual features rather than deeper contextual understanding. This limitation is expected, especially for models like TF-IDF + SVM, which do not capture semantic nuance beyond word distributions.

Additionally, some errors were caused by domain-specific expressions or uncommon vocabulary that were not well represented in the training data. In such cases, the model failed to correctly associate certain words or phrases with their intended sentiment.

We also observed that preprocessing choices impacted error patterns. For instance, removing stopwords and punctuation sometimes led to loss of important contextual cues (such as negation or emphasis), which negatively affected classification in certain cases. Conversely, fully cleaned data improved consistency but occasionally oversimplified the text.

From this analysis, we conclude that the model primarily struggles with:

- Neutral and ambiguous sentiment
- Context-dependent language (e.g., sarcasm, irony)
- Loss of semantic nuance due to preprocessing
- Rare or domain-specific expressions

These findings highlight potential areas for future improvement, such as incorporating contextual embeddings (e.g., transformer-based models), enhancing preprocessing strategies, or using more advanced techniques for handling negation and sarcasm.

Model Deployment

In the final stage of the project, we developed and deployed a complete inference pipeline to make our sentiment classification models accessible in a production-like environment. This stage included building a RESTful API and integrating it with an interactive user interface.
To serve the trained models, we implemented an API using FastAPI. The API was designed to handle real-time predictions through a /predict endpoint, which accepts a JSON request containing the input text and the selected model. The response returns the predicted sentiment, a confidence score, and the probability distribution across all sentiment classes (positive, negative, neutral).
The deployment pipeline includes a preprocessing layer that mirrors the same preprocessing steps used during training, ensuring consistency between training and inference. Incoming text is first cleaned using the predefined preprocessing function, then transformed into numerical features using the saved TF-IDF vectorizer. These features are passed to the trained SVM model to generate predictions.
To support flexibility, multiple models were loaded into the API service, allowing users to choose between different configurations (e.g., fully cleaned schema vs. schema with stopwords retained). The system was designed to handle both probability-based models and decision-function-based models (such as SVM), where scores are converted into probabilities using a softmax function.
For user interaction, we built a lightweight frontend application using Streamlit. The interface allows users to input custom text, select a model, and optionally compare predictions from multiple models simultaneously. The application communicates with the API via HTTP requests and displays the results in a clear and interactive format, including sentiment labels, confidence scores, and visualized probability distributions.
This deployment setup demonstrates a complete end-to-end pipeline, starting from raw user input and ending with interpretable sentiment predictions. It ensures reproducibility, scalability, and usability, making the system suitable for real-world applications and further extensions.

SOTA Model Comparison

In this phase we started by collecting category stats like: how many categories do we have, what are they, and how many records of data each of them have. Then the step of importing a SOTA model came to process and the model of choice was DISTILBERT model from Hugging Face imported with the ‘transformers’ python library and fed with the exact same dataset that was used with our optimized SVM model and the results were then combined with the predictions from the SVM model after adding the category column from the original dataset so that we can split results by category later and along with the ground truth label column we added before. The next step was the act of splitting the results by category and calculating the requested metrics (accuracy and macro f1 score), we used pandas ‘.explode()’ function to split each record’s category tags into multiple duplicates each carrying a category tag from the original tags list of the record, from here the metrics calculation carried on, we calculated models’ accuracies and f1 scores (optimized SVM and SOTA) for each category and added the sample count applied in each category in addition to a column indicating which model performed better in each category, and finally saved the results to a csv and moved to visualization.

Quantitative Results

Lexicon Models Metrics:

Model	Accuracy	Precision	Recall	F1-score
Afinn Fully Cleaned	0.4526	0.4384	0.4526	0.38
Afinn no Lemmatization	0.45	0.4406	0.4526	0.3816
Afinn with Stopwords and no Punctuation	0.4518	0.4428	0.4518	0.3799
Vader Fully Cleaned	0.4414	0.4429	0.4414	0.3649
Vader no Lemmatization	0.4393	0.4203	0.4394	0.3617
Vader with Stopwords and no Punctuation	0.4490	0.4407	0.4490	0.3751

Machine Learning Models Metrics:

SVM:

Model	Accuracy	Precision	Recall	F1-score
SVM TF-IDF Fully Cleaned	0.7366	0.7386	0.7366	0.7374
SVM TF-IDF no Lemmatization	0.7302	0.7314	0.7302	0.7305
SVM with Stopwords and no Punctuation	0.736	0.7384	0.736	0.7367

SVM GloVe Fully Cleaned	0.5909	0.5986	0.5909	0.5929
SVM GloVe no Lemmatization	0.5909	0.5991	0.5909	0.5932
SVM GloVe with Stopwords and no Punctuation	0.5528	0.5794	0.5528	0.5576

Linear Regression:

Model	Accuracy	Precision	Recall	F1-score
Linear Regression TF-IDF Fully Cleaned	0.5132	0.5088	0.5132	0.5097
Linear Regression TF-IDF no Lemmatization	0.5164	0.5133	0.5164	0.5142
Linear Regression with Stopwords and no Punctuation	0.5224	0.5232	0.5224	0.5228
Linear Regression GloVe Fully Cleaned	0.4532	0.6304	0.4532	0.4174
Linear Regression GloVe no Lemmatization	0.4579	0.6386	0.4579	0.4206
Linear Regression GloVe with Stopwords and no Punctuation	0.4368	0.63	0.4368	0.3959

Models chosen for Deployment after Optimization:

Model	Accuracy	Precision	Recall	F1-score
SVM TF-IDF Fully Cleaned	0.7366	0.7386	0.7366	0.7374
SVM TF-IDF with Stopwords and no Punctuation	0.7432	0.7455	0.7432	0.7442

SOTA model comparison and interpretation:

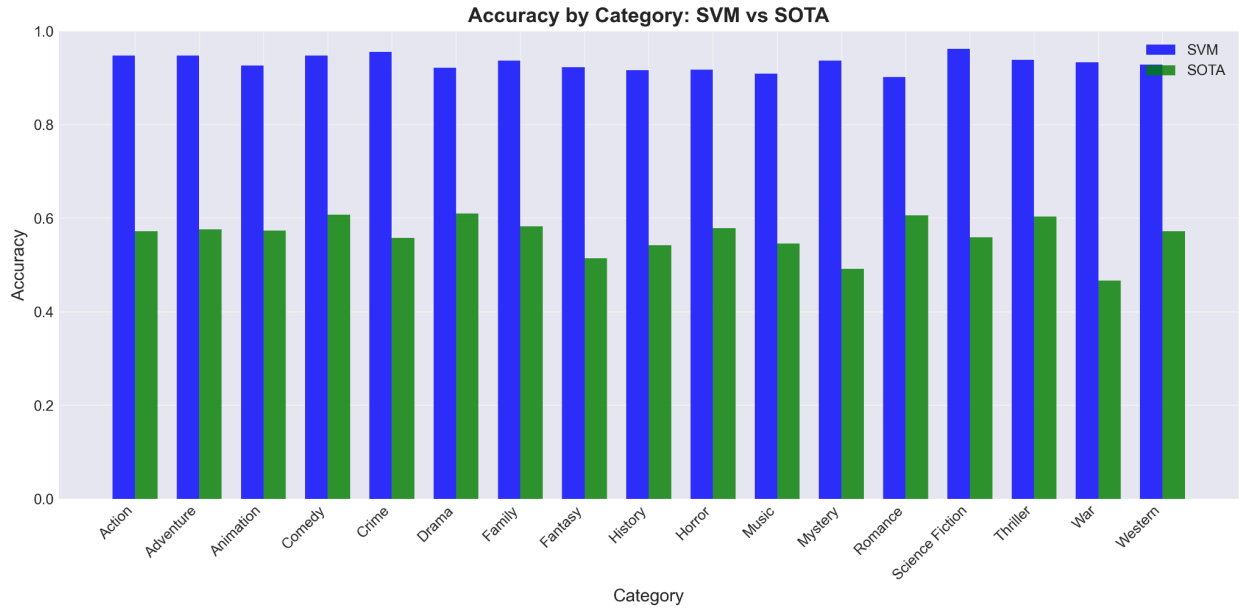
	category	svm_accuracy	svm_f1	sota_accuracy	sota_f1	sample_count	better_accuracy	better_f1
0	Action	0.947735	0.947426	0.571429	0.571241	287	SVM	SVM
1	Adventure	0.947598	0.945084	0.576419	0.574341	229	SVM	SVM
2	Animation	0.926471	0.907180	0.573529	0.537415	68	SVM	SVM
3	Comedy	0.947712	0.947531	0.607843	0.606481	153	SVM	SVM
4	Crime	0.955752	0.955529	0.557522	0.557210	113	SVM	SVM
5	Drama	0.921951	0.921860	0.609756	0.607656	205	SVM	SVM
6	Family	0.936709	0.931388	0.582278	0.577950	79	SVM	SVM
7	Fantasy	0.922330	0.921073	0.514563	0.514517	103	SVM	SVM
8	History	0.916667	0.906250	0.541667	0.520871	24	SVM	SVM
9	Horror	0.917355	0.907774	0.578512	0.558110	121	SVM	SVM
10	Music	0.909091	0.895238	0.545455	0.476190	11	SVM	SVM
11	Mystery	0.936508	0.936492	0.492063	0.491935	63	SVM	SVM
12	Romance	0.901639	0.900327	0.606557	0.587838	61	SVM	SVM
13	Science Fiction	0.962366	0.959559	0.559140	0.552885	186	SVM	SVM
14	Thriller	0.938144	0.938144	0.603093	0.602998	194	SVM	SVM
15	War	0.933333	0.928230	0.466667	0.444444	15	SVM	SVM
16	Western	0.928571	0.918129	0.571429	0.562500	14	SVM	SVM

Our interpretation for these results:

- As we can see none of the models were significantly better at a specific category as the results were close between each category's metrics and the other.
- As for the large difference in performance between the SOTA's and our model's performance we saw two possible reasons for that, the first and the most likely one is the opinionated yet tricky nature of complex human language classification which is represented here in the part where the 'true_label' that we test upon is built on opinions and user ratings and machine learning models just look at human language as 1s and 0s in the end so there is a window for conflict that's unavoidable, the second reason that we

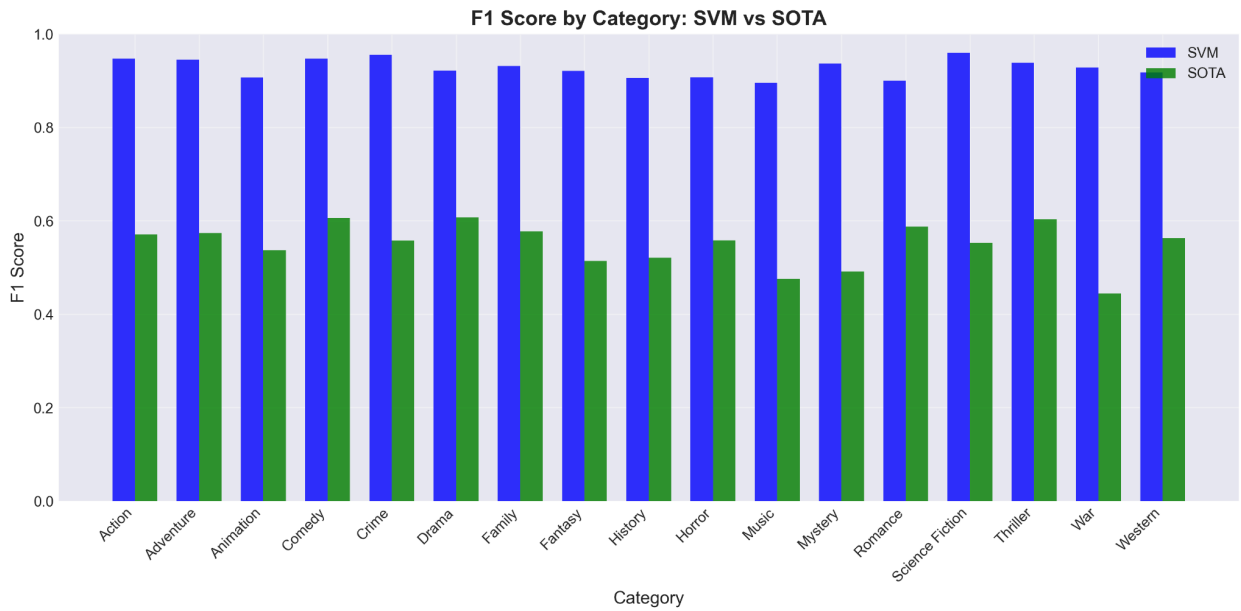
thought of is that our model might be highly optimized and trained on our data so it performs well in terms of movie reviews but might fail to classify some general textual paragraph about war in Iran for example, and on the opposite side the SOTA model might be more generalized and couldn't fit well with our data but it can perform well in more generalized paragraphs.

Visalisation

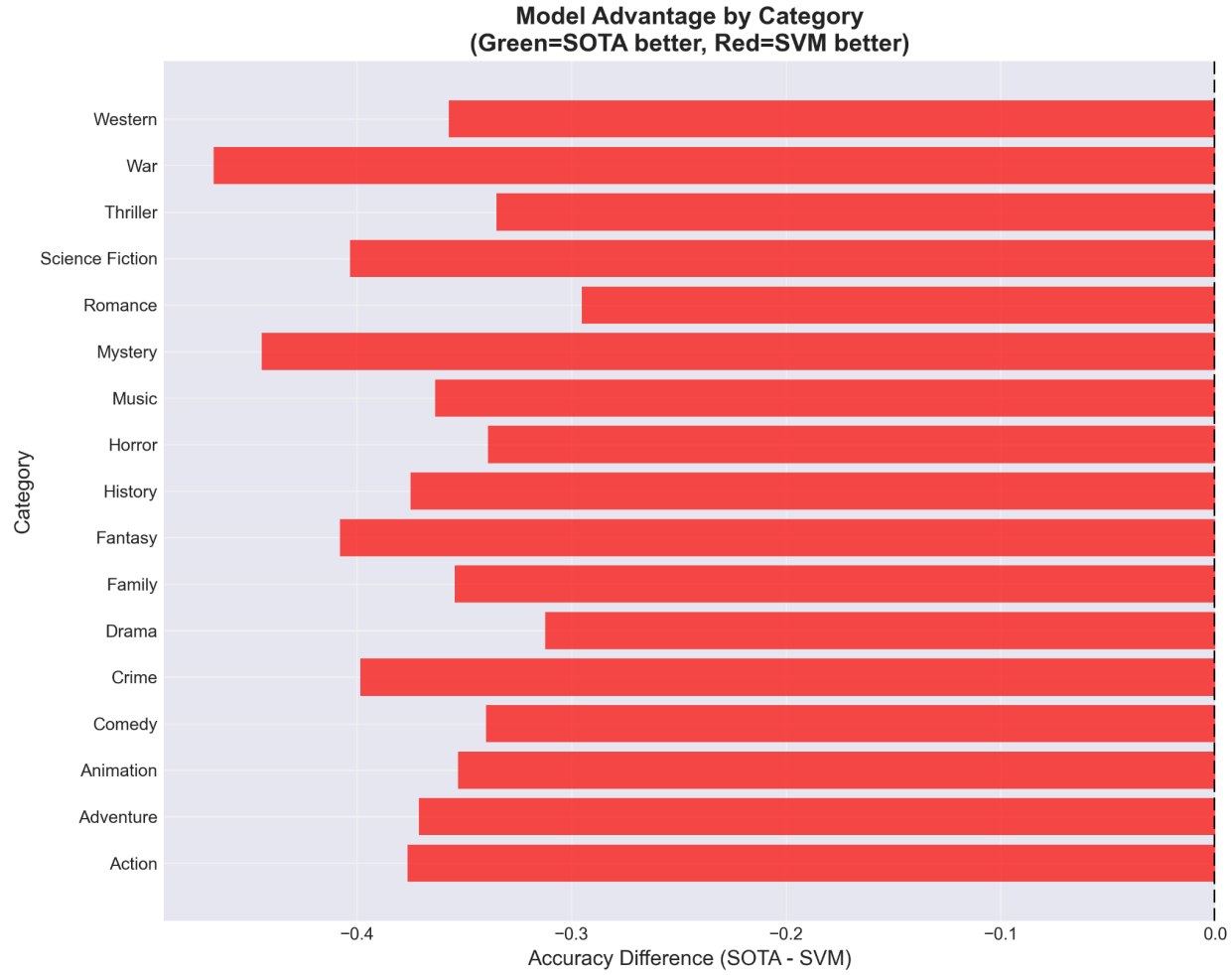


Insights:

- The category with the least accuracy in the SVM model is Romance and in the SOTA model is War and this might be due to a lack of certain words that link polarity with flirting words or war terminologies.
- The top five categories in accuracy in the SVM model are Sci-Fi, Crime, Comedy, Action, and Adventure which may indicate the clearance of feelings in reviews of these movie genres which make it easier for models to classify.



These high f1 scores indicate the model's capability to avoid both false positives (precision) and negatives (recall) effectively.



Conclusion

In conclusion, this project successfully developed a robust sentiment classification system for movie reviews by implementing a comprehensive pipeline covering data preparation, text representation, and advanced modeling. Our methodology compared lexicon-based approaches with supervised machine learning. As well as integrating our models into a production-ready deployment using a FastAPI backend and a Streamlit frontend interface. Along the way, error analysis revealed challenges with ambiguous neutral reviews and implicit contextual sentiment. Furthermore, the comparison with the DistilBERT SOTA model highlighted a performance gap, suggesting that future work should focus on more sophisticated negation handling to bridge the gap between statistical and semantic understanding.